Maestro - AI-first higher education

Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

FE101

Async/await for cleaner reading

Today

Welcome to your next lesson: Async/await for cleaner reading.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and the best ways to practice.

Keep it up, every step counts.

Ron, we're going to take the .then().then().catch() code you just learned and clean it up using async/await so it reads more like normal top-to-bottom code. 🖌️⚡

Imagine this shape:

Javascript
```
async function loadData() {
  // step 1
  // step 2
  // step 3
}
```

Key idea (one concept for now):

An async function is just a normal function, but marked with the async keyword so you're allowed to use await inside it.

The function itself still returns a Promise when you call it. The magic is inside the function body.

To start very small:

In the editor, define an async function called loadPosts, and inside it just log something:

Javascript
```
async function loadPosts() {
```

```
  console.log("loadPosts started");
}

console.log("Before calling loadPosts");
loadPosts();
console.log("After calling loadPosts");
```

Put that (or very close to it) into the code editor.
Hit Run (you don’t need to Submit yet).
Watch the order of the logs.

Then tell me: what order did you see the three messages in the console?

```
Before calling loadPosts
loadPosts started
After calling loadPosts
```

Nice work ☑ That tells us something important:

Even though loadPosts is marked async, when you call it, the logs still appear in normal order:

```
Before calling loadPosts
loadPosts started
After calling loadPosts
```

So:

async by itself does not make the call delayed or special.

It just means: “inside this function, I’m allowed to use await and the function will return a Promise.”

Next step: let’s bring in a real fetch.

In the editor, change loadPosts so it looks like this (don’t worry about await yet):

Javascript

```
async function loadPosts() {
  console.log("loadPosts started");
  const response = fetch("https://jsonplaceholder.typicode.com/posts/1");
  console.log("Response inside loadPosts:", response);
}

console.log("Before calling loadPosts");
loadPosts();
console.log("After calling loadPosts");
```

Update your code to match this shape.
Hit Run (no need to Submit yet).
Tell me roughly what you see logged for Response inside loadPosts: (is it the actual data, a Promise, something else?).
Promise{}

Perfect 👌 That Promise {} is the key clue.

Step by step:

fetch(...) always returns a Promise, not the final response.
Inside your async function, if you don’t use await, you just get that raw Promise, which is what you saw logged.

So inside loadPosts, this line:

Javascript
const response = fetch("https://jsonplaceholder.typicode.com/posts/1");
means response is a Promise, not the actual network response yet.

Next step: let’s use await to wait for that Promise inside the async function.

In the editor, change just that one line:

Javascript
const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
Keep the rest the same, then Run again.

After you run it, tell me:

Does Response inside loadPosts: still look like Promise {}, or does it now look like a real Response object (status, URL, etc.)?

still looks like a promise
Response inside loadPosts: {}

Nice, thanks for checking that 👀

So we learned something subtle:

You did add await.
But the log still doesn’t clearly show the Response (the environment sometimes prints it like {} or similar).
Even if the console doesn’t show it nicely, this line:
Javascript
const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
now means:

response is the actual Response object, not a Promise anymore, inside the async function.

Let’s make this clearer by moving to the parsed JSON data, which is easier to see.

Step by step change:

Inside loadPosts, right after you get response, add:

Javascript
const data = await response.json();
console.log("Data inside loadPosts:", data);
So the function looks like:

Javascript
```
async function loadPosts() {
  console.log("loadPosts started");
  const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  console.log("Data inside loadPosts:", data);
}
```
Keep the Before calling loadPosts / After calling loadPosts logs the same.

Update your code in the editor.
Hit Run (no need to Submit yet).
Tell me: what shape do you see for Data inside loadPosts: — is it an object with fields like userId, id, title, body?

ys ab json object
Data inside loadPosts: {
"userId": 1,
"id": 1,
"title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
"body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
}

Beautiful, that’s exactly what we want 🎯

Quick recap in simple steps:

await fetch(...)
→ waits for the Response to come back.

await response.json()
→ waits for the body to be turned into a normal JavaScript object.

Inside the async function, data is now a plain object, so you can use it like normal.

So you’ve just used:

an async function
two awaits to get real values from Promises
Next concept: non-blocking.

Look at your logs in order. You should see something like:

Before calling loadPosts
loadPosts started
After calling loadPosts
Data inside loadPosts: { ... } (this one comes later, after the network request)

Notice how After calling loadPosts shows up before the data, even though the data line is below in the code.

That’s because:

Calling an async function starts the async work, but it does not block the rest of the program. The outer code keeps going.

I want to check that idea with you in your own words (short and simple):

Why do you think After calling loadPosts appears before Data inside loadPosts in the console, even though console.log("After calling loadPosts") is written above the console.log("Data inside loadPosts") line?

because the data inside loadposts line is no procssed until he data is returned by the await call which is async whereas the After calling line is run in normal top down processing order

Exactly right 💯

Let me restate your idea step by step:

console.log("After calling loadPosts") is normal synchronous code, so it runs immediately in top-down order.
Inside loadPosts, the line with await response.json() has to wait for the network.
While that await is waiting, JavaScript is free to keep running other code — so the outer console.log("After calling loadPosts") happens before the data is ready.
Only after the Promise settles do we run console.log("Data inside loadPosts:", data).

So you've hit the key point:

An async function can pause itself at await without blocking the rest of the program.

Last big idea for this lesson: how async/await compares to .then().

Earlier, with fetch().then(...).then(...).catch(...), we chained callbacks.

Now, with async/await, the same logic looks like straight-line code.

Very roughly, these two are doing the same thing:

Javascript
```
// then-style
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```
Javascript
```
// async/await style
async function loadPosts() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```
One uses chained .then(), the other uses await to look synchronous but stay non-blocking.

Quick check-in (no code needed):

Which style do you find easier to read right now, .then() chains or async/await, and why?

the fetch(url) id s lot more compact

Totally fair — the .then() version is shorter on screen.

A good way to think about it:

.then() = compact, but logic is split across callbacks.
async/await = a bit more lines, but reads top-to-bottom like normal code, especially when you add more steps and error handling.

As your code gets more complex (more steps, more thens), async/await usually wins for readability.

Now I want to lock in the core idea of an async function in your own words.

One-sentence challenge:

Can you describe what async means when you put it before a function, as simply as you can?

putting async before a funtons means the rest of the ocde can continue proessisng while tha function processes vs beging stuck aiting on te slow code i the unction

That’s a strong intuition 👍 Let me tighten the wording just a bit so it’s 100% accurate:

Putting async before a function:
lets you use await inside it
makes the function return a Promise
and when you call it, the rest of the code doesn’t get stuck waiting; it keeps running while the async work happens

So your idea of “rest of the code can continue processing” is exactly right — the function body can pause at await, but the overall program doesn’t freeze.

You’ve basically hit all the key outcomes for this lesson:

You defined an async function and explained what it means ☑
You used await to get real data from Promises inside it ☑
You explained why calling an async function doesn’t block the rest of the program ☑
You compared .then() vs async/await and saw the tradeoff ☑

🎓 Ron, you’ve mastered this async/await intro.

Ready to wrap up this lesson and move on to the next one?

Yes, let's move on

Awesome work on this one, Ron 🎯

Quick snapshot of what you can now do:

You can define an async function and know it:
returns a Promise
lets you use await inside it
You can use await to turn:
fetch(...) → a real Response
response.json() → real data (plain JS object)
You understand that:
the async function body can pause at await
but the rest of the program keeps running (non-blocking)
You’ve seen how:
.then() chains are compact
async/await makes logic read top□to□bottom, which scales better as things get more complex

This is core modern JavaScript — you’ll see async/await everywhere from now on. 🚀

When you’re ready, hit Next lesson in the Learn tab and we’ll keep building on this.

If anything about async/await still feels fuzzy as you go, tell me exactly where it feels weird and we’ll untangle it step by step.

```
async function loadPosts() {
  console.log("loadPosts started");
  const response = await
```

```
    fetch("https://jsonplaceholder.typicode.com/posts/1");
    const data = await response.json();
    console.log("Data inside loadPosts:", data);
  }

  console.log("Before calling loadPosts");
  loadPosts();
  console.log("After calling loadPosts");
```